Lista Circular Simple

- **NODO**

```
namespace ListaCircularSimple
{
   public class Nodo
   {
      // campos o atributos
      public int info { get; set; }
      public Nodo sig { get; set; }
   }
}
```

- **ListaCircularSimple**

```
namespace ListaCircularSimple
{
   public class ListaCircularSimple
   {
      Nodo raiz, nuevo, aux, ptr; // ptr = pointer = puntero = direccion direccion =
numero

      // Método Constructor
      public ListaCircularSimple() { raiz = null; }

      // Método InsertarLIFO
      public void InsertarLIFO(int dato)
      {
         nuevo = new Nodo();
         nuevo.info = dato;
         if (raiz == null)
```

```
    {
        nuevo.sig = nuevo;
        raiz = nuevo;
        aux = nuevo;
    }
    else
    {
        nuevo.sig = raiz;
        aux.sig = nuevo;
        raiz = nuevo;
    }
}
// Método InsertarFIFO
public void InsertarFIFO(int dato)
{
    nuevo = new Nodo();
    nuevo.info = dato;
    if (raiz == null)
    {
        nuevo.sig = nuevo;
        raiz = aux = nuevo;
    }
    else
    {
        aux.sig = nuevo;
        nuevo.sig = raiz;
        aux = nuevo;
    }
```

```
}

// Método Recorrido
public void Recorrido()
{
    ptr = raiz;
    do
    {
        Console.Write("{0},", ptr.info);
        ptr = ptr.sig;
    } while (ptr != raiz);
}

// Método Buscar
public int Buscar(int dato)
{
    int resultado = 0;
    ptr = raiz;
    do
    {
        if (ptr.info == dato)
        {
            resultado = 1; break;
        }
        ptr = ptr.sig;
    } while (ptr != raiz);
    return resultado;
}
```

```
// Método Eliminar
public void Eliminar(int dato)
{
    bool resultado = false;
    if (raiz.info == dato)
    {
        resultado = true;
        aux = raiz;
        while (aux.sig != raiz)
        {
            aux = aux.sig;
        }
        raiz = raiz.sig;
        aux.sig = raiz;

    }
    else
    {
        ptr = aux = raiz;
        do
        {
            if (ptr.info == dato)
            {
                resultado = true;
                aux.sig = ptr.sig;
                break;
            }
```

```
            aux = ptr;
            ptr = ptr.sig;
        } while (ptr != raiz);
    }
    Console.WriteLine("{0} ELIMINADO", resultado ? "SI" : "NO");
    Recorrido();
}

// Método Ordenar
public void Ordenar()
{   // > (Ascendente), < (Descendente)
    ptr = raiz;
    do
    {
        aux = raiz;
        do
        {
            if (ptr.info < aux.info)
            {
                int temp = ptr.info;
                ptr.info = aux.info;
                aux.info = temp;
            }
            aux = aux.sig;
        } while (aux != raiz);
        ptr = ptr.sig;
    } while (ptr != raiz);
}
```

```
    }
```

- **Program**

```
namespace ListaCircularSimple
{
    public class Program
    {
        static void Main(string[] args)
        {
            ListaCircularSimple lista = new ListaCircularSimple();
            // Añadir 5 elementos
            lista.InsertarLIFO(50);
            lista.InsertarLIFO(14);
            lista.InsertarLIFO(18);
            lista.InsertarLIFO(10);
            lista.InsertarLIFO(90);
            // Listar los elementos
            Console.WriteLine("--- LISTADO DE ELEMENTOS ---");
            lista.Recorrido();
            //Ordenar los elementos
            lista.Ordenar();
            // Listar los elementos
            Console.WriteLine("--- LISTADO DE ELEMENTOS ORDENADOS ---");
            lista.Recorrido();
            // Buscar elemento
            int dato;
            do
            {
```

```
            Console.Write("\nDATO A BUSCAR (-1 FIN):");
            dato = Int32.Parse(Console.ReadLine());
            Console.WriteLine("{0} {1} ENCONTRADO", dato,
                lista.Buscar(dato) == 1 ? "SI" : "NO");
        } while (dato != -1);
        // Eliminar elemento
        do
        {
            Console.Write("\nDATO A ELIMINAR (-1 FIN):");
            dato = Int32.Parse(Console.ReadLine());
            lista.Eliminar(dato);
        } while (dato != -1);

        Console.ReadLine();
    }
}
```

Lista Circular Doble

- **<u>Nodo:</u>**

```
public class Nodo
{
    public int info { get; set; }
    public Nodo ant { get; set; }
    public Nodo sig { get; set; }
}
```

- **<u>ListaCircularDoble:</u>**

```
namespace ListaCircularDoble
{
    public class ListaDoble
    {
        Nodo raiz1, raiz2, ptr, aux, nuevo;
        // Costructor
        public ListaDoble()
        {
            raiz1 = raiz2 = null;
        }

        // INSERTARLIFO
        public void InsertarLIFO(int dato)
        {
            nuevo = new Nodo();
            nuevo.info = dato;
            if (raiz1 == null)
            {
                nuevo.sig = nuevo.ant = null;
```

```
        raiz1 = raiz2 = nuevo;
        aux = nuevo;
    }
    else
    {
        nuevo.sig = raiz1;
        nuevo.ant = raiz2;
        raiz1.ant = nuevo;
        raiz1 = nuevo;
        aux.sig = nuevo;
    }
}


// INSERTARFIFO
public void InsertarFIFO(int dato)
{

    nuevo = new Nodo();
    nuevo.info = dato;
    if (raiz2 == null)
    {
        nuevo.sig = nuevo.ant = null;
        raiz1 = raiz2 = aux = nuevo;
    }
    else
    {
        nuevo.sig = raiz1;
```

```
        nuevo.ant = raiz2;
        raiz2.sig = nuevo;
        raiz2 = nuevo;
        aux = nuevo;
    }
}



// RECORRIDO
public void Recorrido()
{
    ptr = raiz1;
    Console.WriteLine("\n ---- LISTADO DE ELEMENTOS ----");
    do
    {
        if (ptr == null)
        {
            Console.WriteLine("Vacía");
        }
        else
        {
            Console.Write("{0},", ptr.info);
            ptr = ptr.sig;
        }
    } while (ptr != raiz1);
    Console.WriteLine("");
}
```

```
// BUSCAR
public bool Buscar(int dato)
{
    bool encontrado = false;
    ptr = raiz1;
    do
    {
        if (ptr.info == dato)
        {
            encontrado = true;
        }
        else
        {
            ptr = ptr.sig;
        }
    } while (ptr != raiz1 && !encontrado);
    return encontrado;
}



// ELIMINAR
public bool Eliminar(int dato)
{

    ptr = raiz1;
    bool encontrado = false;

    // si es lista vcia mandar console
```

```
if (ptr == null) encontrado = false;

else if (raiz1 == raiz2 && raiz1.info == dato)
{
   raiz1 = raiz2 = null;
   return true;
}

else if (raiz1.info == dato)
{
   encontrado = true;
   aux = raiz1;
   do
   {
      aux = aux.sig;
   } while (aux != null && aux.sig != raiz1);
   raiz1 = raiz1.sig;
   raiz1.ant = aux.ant;
   aux.sig = raiz1;
}
else if (raiz2.info == dato)
{
   encontrado = true;
   raiz2 = raiz2.ant;
   raiz2.sig = raiz1;
   raiz1.ant = raiz2;
}
else
```

```
    {
        // Búsqueda
        do
        {
            if (ptr.info == dato)
            {
                encontrado = true;
            }
            else { ptr = ptr.sig; }
        } while (ptr != raiz1 && !encontrado);

        if (encontrado)
        {
            ptr.ant.sig = ptr.sig;
            ptr.sig.ant = ptr.ant;
        }
    }
        return encontrado;
}

// ORDENAR
public void Ordenar()
{
    Nodo ptr1, ptr2;
    ptr1 = raiz1;
    do
    {
        ptr2 = raiz1;
```

```
            do
            {
                if (ptr1.info > ptr2.info)
                {
                    int aux = ptr1.info;
                    ptr1.info = ptr2.info;
                    ptr2.info = aux;
                }
                ptr2 = ptr2.sig;
            } while (ptr2 != raiz1);
        } while (ptr1 != raiz1);
            ptr1 = ptr1.sig;
        }
    }
```

- **Program**

```
namespace ListaCircularDoble
{
    public class Program
    {
        static void Main(string[] args)
        {
            ListaDoble milista = new ListaDoble();
            Random rnd = new Random();

            for (int i = 0; i < 5; i++)
            {
                milista.InsertarFIFO(rnd.Next(10, 100));
            }
```

```
// Imprimir
milista.Recorrido();

// Ordenar
milista.Ordenar();
Console.Write("DATOS ORDENADOS");
milista.Recorrido();

int dato = 0;
bool res = false;
string msg = "";
//Buscar
Console.WriteLine("Método Buscar:  Para finalizar escribe -1");
while (dato != -1)
{
   dato = Int32.Parse(Console.ReadLine());
   res = milista.Buscar(dato);
   msg = res ? "Se encontró." : "No se encontró. Intenta nuevamente.";

   Console.WriteLine("Dato {0} " + msg, dato);
}

//Eliminar
Console.WriteLine("Método Eliminar:  Para finalizar escribe -1");
do
{
   dato = Int32.Parse(Console.ReadLine());
```

```
            res = milista.Eliminar(dato);


            msg = res ? "Se eliminó." : "No se eliminó.";
            Console.WriteLine("Dato {0} " + msg, dato);
            milista.Recorrido();


        } while (dato != -1);
    }
}
```